SESSION 6

kubectl-ai

for Intelligent Ops

AI-Powered Kubernetes Management · Natural Language Ops · Intelligent Troubleshooting

kubectl-ai | Kubernetes | AI/ML | DevOps | LLMs

⎈ Kubernetes + AI

Session Agenda

01

Overview of kubectl-ai

What it is, why it matters, architecture & providers

02

AI-Assisted Kubernetes Troubleshooting

Diagnosing pods, nodes, network issues & crash loops

03

Natural Language Queries for Ops Insights

Conversational cluster management & automation

04

Installation & Configuration

Setup, LLM providers, MCP integration

05

Use Cases & Demo Scenarios

Real-world examples and best practices

06

Security & Comparison

Safety controls, alternatives, and future roadmap

The Kubernetes Complexity Problem

Steep Learning Curve

100s of commands, flags, and YAML syntax to memorize

Complex Troubleshooting

Root cause analysis across pods, nodes, and networks

YAML Sprawl

Writing and maintaining dozens of manifest files

Slow Context Switching

Finding docs mid-incident wastes critical time

Why Teams Struggle

67%

of teams say K8s complexity slows delivery

45%

of incidents caused by misconfiguration

3hrs

avg. time to resolve a production issue

What is kubectl-ai?

kubectl-ai is an open-source AI-powered kubectl plugin by GoogleCloudPlatform that translates natural language into precise Kubernetes operations — eliminating the need to memorize complex commands, flags, and YAML syntax.

Natural Language

Type what you want in plain English. kubectl-ai interprets intent and executes the right commands.

Multi-LLM Support

Works with Gemini, OpenAI, Azure OpenAI, Grok, Vertex AI, and local models via Ollama/llama.cpp.

kubectl Plugin

Integrates natively as a kubectl plugin. Also runs as an MCP server for AI client integrations.

Ref: github.com/GoogleCloudPlatform/kubectl-ai — Official Repository

kubectl-ai Architecture

User Input
Natural Language
Prompt / Query

AI Interface
LLM Processing
& Context Parsing

K8s Adapter
kubectl Context
& Namespace Check

Command Exec
Safe Execution
With Confirmation

Supported LLM Providers

- Google Gemini
- OpenAI GPT
- Vertex AI
- Ollama (Local)

MCP Server Mode

kubectl-ai can act as an MCP (Model Context Protocol) server, exposing kubectl tools to AI clients like Claude, Cursor, or VS Code.

- Standard HTTP endpoint: localhost:9080/mcp
- Enhanced mode with filesystem, web search & DB tools
- Full cluster operations via conversational AI clients

Ref: kubectl-ai Architecture — github.com/GoogleCloudPlatform/kubectl-ai#mcp-server

Key Features of kubectl-ai

Natural Language Interface

Ask questions and issue commands in plain English. No need to memorize kubectl flags or YAML schemas.

Interactive Multi-turn

Engage in follow-up conversations. Refine commands, reprompt, or explore alternatives in a single session.

Safe Confirmation Mode

Previews every command before execution. Requires explicit approval for resource-modifying operations.

k8s-bench Benchmarking

Built-in tool to evaluate and compare AI model performance on Kubernetes-specific tasks.

Terminal & Web UI

Switch between terminal mode and a rich web-based interface (uiType: web) for visual interaction.

Unix Pipeline Support

Pipe YAML manifests through kubectl-ai for AI-powered transformations and apply workflows.

Installation & Configuration

Installation Methods

Bash Script (Recommended)

```
curl -sSL https://raw.githubusercontent.com/
GoogleCloudPlatform/kubectl-ai/main/install.sh | bash
```

Binary Download & Install

```
tar -zxvf kubectl-ai_Darwin_arm64.tar.gz
chmod a+x kubectl-ai && sudo mv kubectl-ai /usr/local/bin/
```

Krew Plugin Manager

```
kubectl krew install kubectl-ai
```

NixOS

```
environment.systemPackages = with pkgs; [ kubectl-ai ];
```

Configuration (config.yaml)

```yaml
# LLM Provider Config
llmProvider: "gemini"
model: "gemini-2.5-pro-preview"

# Tool Settings
skipPermissions: false
maxIterations: 20

# MCP Config
mcpServer: false
mcpClient: false

# UI Settings
uiType: "terminal"  # or "web"
uiListenAddress: "localhost:8888"

# Kubernetes
kubeconfig: "~/.kube/config"
```

Ref: github.com/GoogleCloudPlatform/kubectl-ai — Installation Guide

PART 02

AI-Assisted Kubernetes Troubleshooting

Diagnosing failures • CrashLoopBackOff • OOMKilled • Pending Pods • Network Issues

Troubleshooting Pod Failures with kubectl-ai

Common Pod Issues

CrashLoopBackOff

App crash, missing config, OOM kill

OOMKilled

Memory limit exceeded

Pending

No schedulable node, resource quota

ImagePullBackOff

Wrong image tag or registry creds

Init:Error

Init container failure

kubectl-ai Troubleshooting Commands

```
$ kubectl ai "find all failed pods and explain what went wrong"
```

→ Lists and diagnoses failing pods

```
$ kubectl ai "why is the payment-service pod crashing?"
```

→ Root cause for specific pod

```
$ kubectl ai "show pods with high memory usage and check OOM events"
```

→ Detects OOMKilled scenarios

```
$ kubectl ai "find pods stuck in pending and suggest fixes"
```

→ Scheduling issue resolution

Node & Cluster Health Diagnosis

Live Demo Scenario: Node Health Check

```
$ kubectl ai --model gemini-2.0-flash
>>> show me the health status of all nodes and highlight any issues

→ Running: kubectl get nodes
→ Running: kubectl describe node worker-1
→ Running: kubectl describe node worker-2
→ Running: kubectl describe node worker-3

✅ worker-1: Ready — CPU 45%, Memory 62%
✅ worker-2: Ready — CPU 38%, Memory 71%
⚠️  worker-3: Ready but SchedulingDisabled — No new pods can be scheduled
```

🔍

Resource Pressure

kubectl-ai detects CPU, memory & disk pressure conditions on nodes and explains recommended remediation steps.

🚫

Cordoning & Draining

Ask kubectl-ai to safely cordon a node, drain workloads, and confirm healthy redistribution before maintenance.

📈

Capacity Analysis

Query cluster-wide resource utilization and get AI-generated recommendations for right-sizing nodes.

Network & Service Troubleshooting

Service Endpoint Diagnosis

```
kubectl ai "why can't my frontend reach the backend-svc?"
```

Checks Service selectors, Endpoint readiness, NetworkPolicy rules, and DNS resolution. Suggests corrective YAML patches.

Ingress & Load Balancer Issues

```
kubectl ai "the ingress for api.example.com returns 502, diagnose it"
```

Inspects Ingress rules, backend service health, TLS configuration, and controller logs. Provides fix recommendations.

NetworkPolicy Conflicts

```
kubectl ai "show all NetworkPolicies affecting the payments namespace"
```

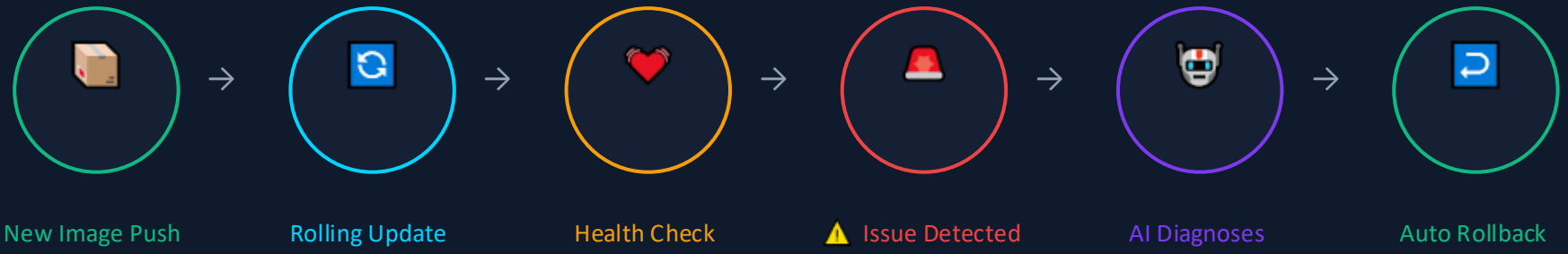
Lists and explains policies, identifies unintended restrictions, and suggests minimal allow rules.

DNS Troubleshooting

```
kubectl ai "CoreDNS is returning SERVFAIL for external domains"
```

Checks CoreDNS pods, ConfigMap, upstream resolvers, and pod resolv.conf. Generates corrected config.

Deployment & Rollout Troubleshooting




```
kubectl ai "my nginx deployment is stuck at 50% — why?"
```

→ Checks replica status, pod conditions, image availability, readiness probes, and resource quotas.

```
kubectl ai "rollback the payments deployment to the previous version"
```

→ Executes kubectl rollout undo, confirms rollback health, and shows diff between revisions.

```
kubectl ai "set memory limit to 512Mi for all pods in production namespace"
```

→ Generates batch patch commands with preview before applying resource constraint updates.

```
kubectl ai "scale the api-gateway to 10 replicas if CPU > 80%"
```

→ Creates or updates HPA policy with conditional scaling rules and validates the configuration.

PART 03

Natural Language Queries for Ops Insights

Conversational cluster management · Ops intelligence · Resource visibility · Automation

Natural Language Queries — Examples

📊 Observability

```
$ kubectl ai "Show resource utilization across all namespaces"
```

```
$ kubectl ai "List pods sorted by memory consumption"
```

```
$ kubectl ai "Which deployments haven't been updated in 30 days?"
```

⚙️ Configuration

```
$ kubectl ai "Create a ConfigMap for database settings"
```

```
$ kubectl ai "Update the HPA min replicas to 3 for api-svc"
```

```
$ kubectl ai "Add an environment variable to the web deployment"
```

🔒 Security & RBAC

```
$ kubectl ai "List all service accounts with cluster-admin role"
```

```
$ kubectl ai "Show pods running as root user"
```

```
$ kubectl ai "Audit RBAC permissions for the dev namespace"
```

🔄 Lifecycle

```
$ kubectl ai "Restart all deployments in the staging namespace"
```

```
$ kubectl ai "Delete completed and failed jobs older than 7 days"
```

```
$ kubectl ai "Set image pull policy to Always for worker pods"
```

AI-Powered Manifest Generation

From Natural Language → YAML Manifest

```
$ kubectl ai "create an nginx pod in namespace foo"
```

```
✨ Attempting to apply:
---
apiVersion: v1
kind: Namespace
metadata:
  name: foo
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: foo
spec:
  containers:
  - name: nginx
    image: nginx:latest

Would you like to apply this?
[Reprompt / Apply / Don't Apply]
```

Unix Pipeline Workflow

```
cat foo-deployment.yaml | kubectl ai "change replicas to 5" --raw
| kubectl apply -f -
```

Pipe existing manifest, modify with AI, apply directly

```
kubectl ai "create a service for nginx with LoadBalancer" --
require-confirmation=false
```

Skip confirmation for automation scripts

```
kubectl get --raw /openapi/v2 > swagger.json
kubectl ai --use-k8s-api "create CRD for analytics"
```

Use cluster's OpenAPI spec for accurate CRD generation

Ops Automation with Natural Language

🚀 CI/CD Integration

kubectl-ai can be embedded in CI/CD scripts to validate manifests, update deployments, and verify rollout health — all via natural language commands in shell scripts.

```
#!/bin/bash
kubectl ai "deploy image:v2.1 to production and verify all pods are ready"
--require-confirmation=false
```

📋 Scheduled Ops Tasks

Automate routine tasks like cleaning up stale resources, rotating secrets, and generating capacity reports through conversational commands.

```
# Cron job example
kubectl ai "delete completed jobs older than 24h in all namespaces" --quiet
```

🔔 Alert-Driven Response

Trigger kubectl-ai with alert payloads to auto-diagnose issues. Feed alert context directly as a prompt to get instant triage and remediation steps.

```
kubectl ai "pod my-service-pod in production is OOMKilled, analyze and
suggest fix"
```

📊 Compliance Reporting

Generate compliance reports by querying cluster state. Ask about privileged containers, missing resource limits, or exposed secrets in a single sentence.

```
kubectl ai "generate a security audit report: privileged pods, root
containers, missing limits"
```

MCP Server & AI Client Integration

kubectl-ai as an MCP Server exposes full Kubernetes operations to any MCP-compatible AI client — including Claude Desktop, Cursor, and VS Code — over a standard HTTP endpoint at localhost:9080/mcp

Start MCP Server:

```
kubectl-ai --mcp-server    # starts at localhost:9080/mcp
kubectl-ai --mcp-server --external-tools   # enhanced mode with filesystem, web, DB tools
```

Supported AI Clients

Claude Desktop

Use Claude to manage your cluster through natural conversation, with full access to kubectl operations.

Cursor IDE

Manage K8s resources directly inside your code editor. Link deployments to code changes.

VS Code

Integrated K8s management within your development environment without context switching.

Custom MCP Client

Any MCP-compatible client can connect and interact with the full Kubernetes API surface.

Local LLM Support — Private & Offline Ops

Why Local LLMs?

- No cluster data sent to external APIs
- Zero API cost — run fully offline
- Low-latency for air-gapped environments
- Meets enterprise data sovereignty rules

Supported Local Models via Ollama

LLaMA 3

Meta's flagship open model, excellent K8s reasoning

Mistral

Fast, efficient, great for structured output

Phi-3

Microsoft's compact but capable model

CodeLlama

Optimized for YAML/config generation

Gemma

Google's lightweight open model (also in k8s)

llama.cpp

Direct runtime for GGUF format models

Config for Ollama (local)

```yaml
model:
  provider: openai
  openai:
    base_url: http://localhost:11434/v1
    api_key: sk-ollama-placeholder
    model: mistral
```

Security & Safety Controls

⚠️ Safety Note: kubectl-ai can generate incorrect commands or misinterpret instructions. Users should have a basic Kubernetes understanding before using the tool in production.

🔍 Confirmation-First Mode

Default behavior: kubectl-ai ALWAYS shows the generated command/manifest and waits for your approval before executing any resource-modifying operation (--require-confirmation=true).

👁 Read-Only by Default

kubectl-ai operates read-only unless explicitly triggered to write. Destructive actions require explicit confirmation, reducing risk of accidental cluster changes.

🏠 Local Model Option

Organizations with strict data policies can use Ollama/llama.cpp to ensure zero cluster data leaves the network. No API keys, no external calls.

🔐 RBAC Awareness

kubectl-ai respects the permissions of the configured kubeconfig user. It checks namespace and access rights before executing commands.

📝 Audit Trail

All AI-generated commands are visible in terminal history. The --quiet flag suppresses output for scripts while maintaining log trails.

⚙️ skipPermissions Flag

For trusted automation scripts, skipPermissions: true can disable confirmation prompts. Use with extreme caution — only in controlled pipelines.

kubectl-ai vs Alternatives

Tool	NL Interface	LLM Providers	MCP Support	Local Model	K8s-Native	Best For
kubectl-ai (GCP)	✅ Full	Gemini, OpenAI, Azure, Grok, Ollama	✅ Yes	✅ Ollama	✅ Yes	Full-stack K8s ops
kubectl-ai (sozercan)	✅ Full	OpenAI, Azure	❌ No	❌ No	✅ Yes	Manifest generation
k8sgpt	⚠️ CLI	OpenAI, Azure, Local	❌ No	✅ Yes	✅ Yes	Cluster analysis
HolmesGPT	✅ Chat	OpenAI, Azure	✅ k9s plugin	❌ No	⚠️ Partial	Incident triage
KoPylot	⚠️ CLI	OpenAI (Davinci)	❌ No	❌ No	✅ Yes	Auditing & diagnose

✅ = Full Support ⚠️ = Partial ❌ = Not Supported

Best Practices & Pro Tips

01

Use Concise, Clear Prompts

Short, specific prompts yield better results. Instead of 'fix things', say 'why is pod X in namespace Y crashing?'

02

Enable Verbose Mode for Debugging

Use --verbose flag to see the raw AI request/response logs. Essential for troubleshooting LLM misinterpretations.

03

Always Review Before Apply

Never skip the confirmation step in production. Read the generated YAML carefully — AI can produce unexpected manifests.

04

Use k8s-bench to Pick Your Model

Run the built-in benchmarking tool to compare LLM performance on your specific K8s task types before committing to a provider.

05

Leverage Multi-turn for Complex Tasks

Break complex operations into steps. Use reprompt to refine generated manifests rather than writing from scratch.

06

Combine with Monitoring Tools

Pipe kubectl-ai with tools like k9s, Lens, or Prometheus queries for richer context when diagnosing production issues.

Session 7 — Key Takeaways

AI-Native kubectl Plugin

kubectl-ai bridges natural language and Kubernetes, eliminating the need to memorize hundreds of commands and YAML schemas.

Intelligent Troubleshooting

Diagnose CrashLoopBackOff, OOMKilled, network issues, and deployment failures conversationally with full context awareness.

Natural Language Ops

Query cluster state, generate manifests, manage RBAC, and automate tasks through plain English — from terminal or AI clients via MCP.

Secure by Design

Confirmation-first execution, read-only defaults, RBAC awareness, and local LLM support for air-gapped/compliant environments.